Specification

Specification · Filter catalog: cheap AND in-stock · Build with Waqar

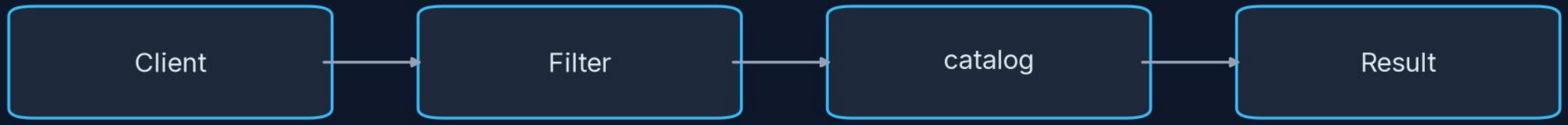
Today's Hook

- Problem: Filter catalog: cheap AND in-stock
- Pattern: Specification — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Filter catalog: cheap AND in-stock
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Specification: Filter catalog: cheap AND in-stock

Real-World Scenario

- Running example: Filter catalog: cheap AND in-stock
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.enterprise.specification;

import com.buildwithwaqar.patterns.enterprise.specification.*;
/** Problem: compose catalog filters (price AND stock) without nested ifs. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Specification problem: catalog filter ===");
    Specification<Product> rule = new PriceSpec(1000).and(new InStockSpec());
    for (var p : new Product[]{new Product("mug",500,true), new Product("tv",50000,true), new Product("mug",500,false)})
      System.out.println(p + " => " + rule.test(p));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.enterprise.specification;

import java.util.function.Predicate;

public interface Specification<T> extends Predicate<T> {
    default Specification<T> and(Specification<T> other) {
        return t -> this.test(t) && other.test(t);
    }

    default Specification<T> or(Specification<T> other) {
        return t -> this.test(t) || other.test(t);
    }

    default Specification<T> not() { return t -> !this.test(t); }
}

record Product(String name, int priceCents, boolean inStock) {}

final class PriceSpec implements Specification<Product> {
    private final int max;
    PriceSpec(int max) { this.max = max; }
    public boolean test(Product p) { return p.priceCents() <= max; }
}
```

Solution (2)

```
final class InStockSpec implements Specification<Product> {
    public boolean test(Product p) { return p.inStock(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.specification;

import com.buildwithwaqar.patterns.enterprise.specification.*;
/** Problem: compose catalog filters (price AND stock) without nested ifs. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Specification problem: catalog filter ===");
    Specification<Product> rule = new PriceSpec(1000).and(new InStockSpec());
    for (var p : new Product[]{new Product("mug",500,true), new Product("tv",50000,true), new Product("mug",500,false)})
      System.out.println(p + " => " + rule.test(p));
  }
}
```

Walkthrough Output

```
=== Specification problem: catalog filter ===
Product[name=mug, priceCents=500, inStock=true] => true
Product[name=tv, priceCents=50000, inStock=true] => false
Product[name=mug, priceCents=500, inStock=false] => false
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Specification.

Related Patterns

- Contrast Specification with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Specification → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.